

Reviewer Pack — Roe-Cover Multiplicity Lower Bound from Asymptotic Dimension...

Entry 3035afde3273 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 10:34:31 UTC

Roe-Cover Multiplicity Lower Bound from Asymptotic Dimension Growth in Tensor-Powered Equality Gadgets

Entry ID: 3035afde3273 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-29 10:34:31 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Lifting (CGL) — Geometric Group Theory / Communication Complexity **Field B** (complexity object): Deterministic Communication Complexity ($D^{\{cc\}}$)

Statement:

For the equality gadget G_{eq} with $|X|=|Y|=2$ and Hamming metric, and its n -fold tensor power $G_{eq}^{\otimes k}$, the asymptotic dimension $asdim_R((X \times Y)^n)$ grows linearly in k for fixed scale $R = \text{diam}(G_{eq})$, and every deterministic protocol Π computing $f \circ G_{eq}^{\otimes k}$ of cost c induces a RoeCover with multiplicity $m_\Pi \geq asdim_R((X \times Y)^{kn}) + 1$; hence $CC(f \circ G_{eq}^{\otimes k}) \geq \log_2(asdim_R((X \times Y)^{kn}) + 1)$.

Rationale (proposer's reasoning):

This sub-conjecture tests Axiom A2 (Protocol $\rightarrow$ Cover) in conjunction with A3 (Asdim Amplification), using tensor powers of a simple gadget to force coarse geometric growth. The equality gadget has verifiable Property A and admits explicit Folner sequences; its tensor powers amplify asymptotic dimension due to coarse product structure. A2 predicts that any low-communication protocol must induce a low-multiplicity cover, but if $asdim_R$ grows with k , then so must the minimal cover multiplicity, forcing a communication lower bound. The conjecture targets the core mechanism of CGL: lifting query complexity via coarse dimension.

Taxonomy category: LIFTING (status at proposal time:)

Framework membership: framework fw_6997a27304, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 273bdce54f2022c4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Asymptotic dimension growth vs. multiplicity in tensor-powered equality gadgets

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - coarse geometric lifting AND deterministic communication complexity
- asymptotic dimension growth AND tensor powered equality gadgets - Roe-Cover multiplicity
AND geometric group theory IN communication complexity protocols

Top relevant hits considered: - [http://arxiv.org/abs/2506.12420v1] Deterministic Lifting Theorems for One-Way Number-on-Forehead Communication - [http://arxiv.org/abs/1803.07099v3] Varieties of coarse spaces - [http://arxiv.org/abs/1903.06084v8] Lifting coarse homotopies - [http://arxiv.org/abs/2003.01308v1] Dislocation pipe diffusion and solute segregation during the growth of metastable GeSn - [http://arxiv.org/abs/1705.00570v1] High pressure floating-zone growth of perovskite nickelate LaNiO₃ single crystals - [http://arxiv.org/abs/2405.12883v2] Asymptotic analysis at any order of Helmholtz's problem in a corner with a thin layer: an algebraic approach - [http://arxiv.org/abs/1006.5752v1] Solomon's induction in quasi-elementary groups - [http://arxiv.org/abs/cs/0111062v2] One-way communication complexity and the Neciporuk lower bound on formula size

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import product

# Define constants and utilities
X = [0, 1]
Y = [0, 1]

def hamming_distance(x, y):
    return sum(xi != yi for xi, yi in zip(bin(x)[2:].zfill(2), bin(y)[2:].zfill(2)))

def tensor_product(G1, G2):
    return [[G1[i][j] * G2[k][l] for k, l in product(range(len(G2)), range(len(G2)))] for i, j in product(range(len(G1)), range(len(G1)))]

def asdim_certify(G, R):
    n = len(G)
    if n == 0:
        return 0
    diameter = max(hamming_distance(i, j) for i, j in product(range(n), repeat=2))
```

```

if diameter > R:
    return float('inf')
return math.ceil(math.log(diameter + 1, 2))

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Define the equality gadget G_eq
    G_eq = [[0, 1], [1, 0]]
    R = hamming_distance(0, 1)

    # Compute asdim_R((X*Y)^n) for increasing k using AsdimCertify on G_eq^{*k}
    results = []
    for n in range(1, 4):
        G_eq_kn = tensor_product(G_eq, G_eq)
        for _ in range(3): # Ensure at least 3 instances per seed
            asdim_value = asdim_certify(G_eq_kn, R)
            if asdim_value == float('inf'):
                return {
                    "metric_name": "asdim_R",
                    "metric_value": asdim_value,
                    "instances_tested": len(results),
                    "conjecture_holds": False,
                    "counterexample": "asdim_infinite"
                }
            results.append(asdim_value)

    # Generate deterministic protocols for AND∘G_eq^{*k n} via decision trees
    def protocol_pullback(G, f):
        if len(G) == 0:
            return []
        if len(G) == 1:
            return [f(G[0])]
        mid = len(G) // 2
        left_protocol = protocol_pullback(G[:mid], f)
        right_protocol = protocol_pullback(G[mid:], f)
        return [left_protocol[i] + right_protocol[j] for i, j in product(range(len(left_protocol)), range(len(right_protocol)))]

    def and_function(x, y):
        return x * y

    m_pi_values = []
    for n in range(1, 4):
        G_eq_kn = tensor_product(G_eq, G_eq)
        for _ in range(3): # Ensure at least 3 instances per seed
            protocol = protocol_pullback([i for i in range(len(G_eq_kn))], and_function)
            m_pi = len(protocol) // (len(G_eq_kn) ** n)
            if m_pi < asdim_value + 1:
                return {
                    "metric_name": "m_pi",
                    "metric_value": m_pi,
                    "instances_tested": len(results),
                    "conjecture_holds": False,
                    "counterexample": f"m_pi<{asdim_value}+1"
                }
            m_pi_values.append(m_pi)

```

```

# Compute mean and standard deviation of metric_value
mean_metric_value = sum(results) / len(results)
std_metric_value = math.sqrt(sum((x - mean_metric_value) ** 2 for x in results) / len(results))

# Compute fraction of seeds where conjecture_holds
support_fraction = sum(1 for m_Π in m_Π_values if m_Π >= asdim_value + 1) / len(m_Π_values)

return {
    "metric_name": "asdim_R",
    "metric_value": mean_metric_value,
    "instances_tested": len(results),
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

# Compute mean/std of metric_value and fraction of seeds where conjecture_holds
results = [run_trial(seed) for seed in seeds]
all_metric_values = [r["metric_value"] for r in results if "metric_value" in r]
all_support_fractions = [r["conjecture_holds"] for r in results if "conjecture_holds" in r]

mean_metric_value = sum(all_metric_values) / len(all_metric_values)
std_metric_value = math.sqrt(sum((x - mean_metric_value) ** 2 for x in all_metric_values) / len(all_metric_values))
support_fraction = sum(all_support_fractions) / len(all_support_fractions)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"m_Π<{asdim_value}+1\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

instances_tested': 0, 'conjecture_holds': False, 'counterexample': 'asdim_infinite'}
TRIAL: {'metric_name': 'asdim_R', 'metric_value': inf, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': 'asdim_infinite'}
TRIAL: {'metric_name': 'asdim_R', 'metric_value': inf, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': 'asdim_infinite'}
TRIAL: {'metric_name': 'asdim_R', 'metric_value': inf, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': 'asdim_infinite'}
TRIAL: {'metric_name': 'asdim_R', 'metric_value': inf, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': 'asdim_infinite'}
TRIAL: {'metric_name': 'asdim_R', 'metric_value': inf, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': 'asdim_infinite'}
TRIAL: {'metric_name': 'asdim_R', 'metric_value': inf, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': 'asdim_infinite'}
TRIAL: {'metric_name': 'asdim_R', 'metric_value': inf, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': 'asdim_infinite'}
TRIAL: {'metric_name': 'asdim_R', 'metric_value': inf, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': 'asdim_infinite'}

```

```

TRIAL: {'metric_name': 'asdim_R', 'metric_value': inf, 'instances_tested': 0, 'conjecture_holds': Fa
TRIAL: {'metric_name': 'asdim_R', 'metric_value': inf, 'instances_tested': 0, 'conjecture_holds': Fa
TRIAL: {'metric_name': 'asdim_R', 'metric_value': inf, 'instances_tested': 0, 'conjecture_holds': Fa

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2933a807.py", line 125, in <module>
    print(f"RESULT: FALSIFIED counterexample=\"m_Π<{asdim_value}+1\" first_failing_seed={first_failing_seed}
    ^^^^^^^^^^^^^
NameError: name 'asdim_value' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, and the critic challenged the conjecture. |
next: Fix the test code to handle the case where asdim_value is not defined and re-run the test.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	groq	llama-3.3-70b-versat	0	0	4499
2	novelty	groq	llama-3.3-70b-versat	0	0	4258
3	novelty	groq	llama-3.3-70b-versat	0	0	4656
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10694
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14726
6	judge	groq	llama-3.3-70b-versat	0	0	4520

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 43352 ms total latency. Provider mix: {'groq': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in *research/audit/3035afde3273.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/3035afde3273.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/3035afde3273.tar.gz (if generated)